

Lab Report: Week 8 Session A

Testing & Validation

Mahmudul Hasan (Student ID: 4125999049)

May 22, 2026

Abstract

This report documents Week 8 Session A on the Smart Water capstone: **29** `pytest` cases, **96%** statement coverage on `src/`, a physical validation module (`src/validation.py`), and integration workflows (Swiss Cheese Model). A hallucination in an edge-case assumption ($P=5$ mm always below I_a for high CN) was corrected after tests failed. Evidence: Figure 1 (`pytest --cov`). Repository: <https://github.com/mahmud456alhasan-debug/smart-water-capstone>.

1 Introduction

After Week 7 implementation, the capstone had only two unit tests (27% coverage). Session A expands testing across runoff, weather, reservoir, flood, validation, and integration layers per the lab guide.

2 Objectives

- Comprehensive unit tests (Exercise 1).
- Swiss Cheese verification layers (Exercise 2).
- Find and fix AI/human test assumptions (Exercise 3).
- Document in `prompt_log.md` (Exercise 4).

3 Exercise 1: Comprehensive tests

3.1 Test inventory

Table 1: Test modules and focus.

File	Coverage
<code>test_runoff.py</code>	Normal storm, $P=0$, $Q \leq P$, invalid CN/ P
<code>test_weather.py</code>	CSV load, missing columns, GREEN/AMBER/RED
<code>test_flood.py</code>	Mask, depth, DEM bounds, monotonicity
<code>test_reservoir.py</code>	Revenue $\approx \$708,849$, storage PASS
<code>test_validation.py</code>	Physical check failures
<code>test_integration.py</code>	Rainfall $\rightarrow$ alert, full workflows

3.2 Coverage results

Before Session 8: **2** passed, **27%** coverage. After: **29** passed in **0.63s**, **96%** on `src/` (201 statements, 8 missed—mostly DEM auto-generate branch and rare optimizer error paths).

```

mahmudul-robotics@robotics-ai:~/Downloads/Software Development/ai_water_lab/capstone$ cd ~/home/mahmudul-robotics/Downloads/Software Development/ai_water_lab/capstone
mahmudul-robotics@robotics-ai:~/Downloads/Software Development/ai_water_lab/capstone$ pytest -q --cov=src --cov-report=term-missing
----- coverage: platform linux, python 3.8.10-final-0 -----
Name                               Stmts   Miss  Cover   Missing
-----
src/flood/__init__.py                0      0   100%
src/flood/unundation.py             43      4    91%   40-43
src/reservoir/__init__.py           0      0   100%
src/reservoir/optimizer.py         74      4    95%   69, 74, 121, 123
src/reservoir/wrapper.py            8      0   100%
src/runoff/__init__.py              0      0   100%
src/runoff/scs_cn.py                31      0   100%
src/validation.py                   24      0   100%
src/weather/__init__.py             0      0   100%
src/weather/alerts.py              13      0   100%
src/weather/load.py                 8      0   100%
-----
TOTAL                               201      8    96%

29 passed in 0.62s
mahmudul-robotics@robotics-ai:~/Downloads/Software Development/ai_water_lab/capstone$

```

Figure 1: Terminal: `pytest -q --cov=src --cov-report=term-missing` — 29 passed, 96% total coverage.

4 Exercise 2: Swiss Cheese Model

Table 2: Verification layers applied.

Layer	Action
1	Manual code review (units, Week 6 reservoir logic)
2	29 unit tests, all passing
3	<code>src/validation.py</code> : $Q \leq P$, storage [50, 200] MCM, flood monotonicity
4	<code>tests/test_integration.py</code> : CSV→alert, DEM→stats, reservoir PASS

5 Exercise 3: Hallucination / assumption fix

Issue: Test assumed 5 mm rainfall always yields zero runoff for CN=95. High CN lowers I_a , so computed $Q \approx 0.34$ mm.

Fix: Use $P=2$ mm with CN=80 for a true below- I_a case.

Lesson: Physical edge cases must use formula-derived I_a , not intuition.

6 Exercise 4: Repository

Updated tests and `src/validation.py` committed to <https://github.com/mahmud456alhasan-debug/smart-water-capstone>. `prompt_log.md` includes Session 8 entries (Appendix B).

7 Discussion

1. Coverage jumped from 27% to 96% by testing weather and reservoir modules, not only runoff stubs.
2. Integration tests catch tab-level workflows without running Streamlit headless.
3. One failing test is valuable—it exposed a weak edge-case assumption, not production code failure.

8 Conclusion

Week 8 Session A met the lab guide: broad pytest suite, >80% coverage, Swiss Cheese layers, documented fix, and terminal evidence. Capstone remains deployable via `streamlit run app/main.py` with validated hydrologic constraints.

A Physical validation: validation.py

src/validation.py (full file)

```
"""Physical validation layer (Swiss Cheese Model - Layer 3)."""

from __future__ import annotations

from typing import Any, Dict, List

import numpy as np

from src.flood.inundation import simulate_flood
from src.reservoir.optimizer import V_MAX, V_MIN, validate_schedule
from src.runoff.scs_cn import scs_runoff_mm

def validate_runoff_mm(p_mm: float, q_mm: float, tol: float = 1e-6) -> Dict[str, Any]:
    """Runoff must be non-negative and not exceed rainfall."""
    ok = (q_mm >= -tol) and (q_mm <= p_mm + tol)
    return {"ok": ok, "p_mm": p_mm, "q_mm": q_mm, "message": "Q <= P" if ok else "Runoff
    ↪ exceeds rainfall"}

def validate_storage_mcm(storage_mcm: float, tol: float = 1e-3) -> Dict[str, Any]:
    ok = V_MIN - tol <= storage_mcm <= V_MAX + tol
    return {
        "ok": ok,
        "storage_mcm": storage_mcm,
        "bounds": (V_MIN, V_MAX),
    }

def validate_flood_monotonic(
    dem: np.ndarray, levels: List[float], tol: float = 1e-9
) -> Dict[str, Any]:
    """Flooded cell count must not decrease as stage rises."""
    counts = []
    for level in levels:
        mask, _ = simulate_flood(dem, level)
        counts.append(int(mask.sum()))
    ok = all(counts[i] <= counts[i + 1] + tol for i in range(len(counts) - 1))
    return {"ok": ok, "wet_counts": counts, "levels": levels}

def validate_reservoir_result(result: Dict[str, Any], tol: float = 1.0) -> Dict[str, Any]:
    """Check wrapper output: PASS validation and plausible revenue."""
    revenue = result.get("total_revenue_usd", 0.0)
    ok_val = result.get("validation") == "PASS"
    ok_rev = 600_000 <= revenue <= 800_000
    return {
        "ok": ok_val and ok_rev,
        "validation": result.get("validation"),
        "total_revenue_usd": revenue,
        "revenue_in_expected_band": ok_rev,
    }
```

B Submitted prompt log (excerpt)

prompt_log.md (Session 8 section)

Week 8 Session A - Testing and Validation

Tool: Cursor Agent | Date: 2026-05-22

Exercise 1: Comprehensive tests

Prompt (from lab guide): Write pytest tests with normal, edge, invalid, and physical
↪ cases; fixtures; target >80% coverage for capstone src/ functions.

Added: tests/test_runoff.py, test_weather.py, test_flood.py, test_reservoir.py,
↪ test_validation.py, test_integration.py, tests/conftest.py.

Hallucination found: Assumed P=5 mm always below Ia for CN=95; high CN lowers Ia so
↪ runoff was non-zero. Fixed test to use P=2 mm, CN=80.

Result: 29 passed; coverage 96% on src/ (pytest -q --cov=src --cov-report=term-missing).

Exercise 2: Swiss Cheese Model

Layer 1 - Code review: Manual review of units in reservoir/flood modules.

Layer 2 - Unit tests: 29 pytest cases.

Layer 3 - Physical validation: src/validation.py ($Q \leq P$, storage bounds, monotonic flood).

Layer 4 - Integration: tests/test_integration.py workflows.

Screenshot

week8_session_a_terminal.png - pytest + coverage table (96%).

Lessons learned (Session 8)

Swiss Cheese layers map cleanly to validation.py plus integration tests. Coverage exposes

↪ untested reservoir and weather modules quickly. Always verify SCS-CN Ia before

↪ assuming zero runoff.